GORGO: Online Tuning for Cross-Region Network-Aware LLM Serving

Anonymous Author(s)
Affiliation
Address
email

Abstract

In cross-region LLM serving, time-to-first-token (TTFT) is highly sensitive to per-request routing decisions. KV-cache reuse across replicas can eliminate much of the prefill computation on multi-turn workloads, but realizing this potential requires jointly balancing cache locality against replica load and network cost. Additionally, public datasets to evaluate these policies do not represent long-context, high prefix-reuse workloads: LMSYS and WildChat average 2 and 2.5 turns per conversation, 0.5 and 2.9k tokens/request, and 5 and 9% intra-user reuse, respectively. To capture the setting where a majority of requests contain reusable prefixes, we evaluate common routing policies such as least-load, prefix-aware, session-affinity, and others on traces from a production inference endpoint featuring multi-turn conversations ($\sim$82 requests/user), 7–45$\times$ longer prompts (21k avg tokens), and 2–6$\times$ higher KV-cache reuse (55%). We evaluate policies on a multi-region GPU cluster where cross-region network latency ranges from 26ms to 466ms across replicas to test GORGO, a policy that jointly optimizes cache reuse, network latency, and queue depth with weights optimized online to minimize p95 TTFT with no offline profiling. GORGO achieves the lowest TTFT at every percentile (p50, p95, p99) across both a tuning window and a held-out evaluation window against all other load balancing policies, highlighting the impact of online joint optimization.

1 Introduction

Modern LLM chat services serve interactive workloads where perceived latency is dominated by time-to-first-token (TTFT). On a single replica, TTFT is the sum of three terms: (i) the network round-trip from the client proxy to the replica, (ii) queueing delay behind in-flight requests, and (iii) prefill computation for the input prompt. Inference engines such as SGLang [Zheng et al., 2024b] and vLLM [Kwon et al., 2023] expose radix-tree prefix caches that let an incoming request skip prefill for any token prefix already cached on the handling replica. When prompts are long and prefixes are widely reused, this saving dominates: a 20,000-token prompt that hits a 90% prefix match has only $\sim$2,000 tokens to prefill, reducing TTFT by an order of magnitude.

The KV-cache reuse opportunity is contingent on routing: a request sent to a cold replica forfeits the saving, requests piled onto a warm replica saturate its queue, and a cross-region warm cache can be slower than a local cold one. The routing layer must balance three terms simultaneously—network distance, current load, and cache state—yet standard heuristics (least-request, least-load, prefix-aware, session-affinity) each optimize one and ignore the others. They suffice for workloads with weak prefix reuse and uniform replica latencies [F5 NGINX, 2024], but interactive long-context traffic breaks both assumptions: chat traces concentrate most tokens in a small set of returning users with substantial prefix overlap, and cross-region replicas have RTTs differing by an order of magnitude.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

The right target can change minute-to-minute as load shifts, and static weights leave performance on the table.

Two further factors complicate empirical evaluation. First, the public chat datasets used in prior LLM-serving evaluations underrepresent long-context multi-turn traffic with reusable prefixes. LMSYS-Chat-1M [Zheng et al., 2024a] and WildChat-4.8M [Zhao et al., 2024] average 470 and 2,900 tokens per request, with intra-user prefix reuse below 10%. A production endpoint we evaluate carries 21,000 tokens per request on average and 53.7% intra-user prefix reuse—a regime where joint cache–network–queue trade-offs dominate TTFT. Second, evaluation methodology matters: a routing policy that wins p50 by herding traffic onto a single warm replica typically loses tail latency once that replica saturates. Reporting only one percentile hides this trade-off.

Contributions. (i) A workload characterization (§3) placing three datasets on the same prefix-reuse and request-length axes; the GLM-5.1 production trace has 7–45$\times$ longer prompts and 2–6$\times$ higher token-weighted reuse than LMSYS-Chat-1M and WildChat-4.8M. (ii) A routing cost model with three variants collectively called GORGO (§4) that score each replica by an additive combination of EWMA network RTT, estimated prefill cost over uncached tokens, and a queue-depth penalty; variants differ only in how the two scalar weights are set (static, per-replica median fit, or Gaussian $(1{+}1)$-ES hill climb on p95 TTFT). (iii) An experimental setup (§6) running each of eight policies on its own three-replica L40S:2 SGLang fleet in Asia, Europe, and US East, with cross-region RTTs spanning 26–466 ms. (iv) Empirical results (§7) on two consecutive 30-minute production-trace windows showing GORGO achieves the lowest TTFT at every percentile (p50, p95, p99) on both a tuning and a held-out evaluation window against seven baselines.

We do not claim any single GORGO variant dominates everywhere: `gorgo-hillclimb` wins all three percentiles in the tuning window, while in the evaluation window `gorgo-static` (deploying the tuning-window learned weights) wins p50 and p95 and `gorgo-hillclimb` wins p99. The W2 p99 margin is at the noise floor of one 30-minute window (§8).

2 Background

Serving primitives. Modern LLM serving stacks are stateful by virtue of the Key–Value (KV) cache and admit new requests at every decoding step via continuous batching [Yu et al., 2022]; vLLM's PagedAttention [Kwon et al., 2023] reduces fragmentation by storing KV blocks in fixed-size pages. KV caching also enables prefix reuse: when multiple requests share an identical prefix, the system can skip prefill for cached tokens, which SGLang's RadixAttention [Zheng et al., 2024b] operationalizes via a per-replica radix trie. **Prefix-aware routing and coordination.** Prefix reuse pays off only if requests with shared prefixes land on a replica that already holds the relevant KV state. Preble [Srivatsa et al., 2024] demonstrates that prefix-aware scheduling raises cache utilization; DLPM [Cao et al., 2025] explores fairness–locality trade-offs. Accurate prefix-aware routing requires tracking per-replica cache state, and the cost of doing so via a centralized coordinator scales with cluster size and KV-cache footprint. **Geo-distributed inference.** Multi-region deployments are driven by availability, locality to global users, and the ability to follow diurnal demand. SkyServe [Mao et al., 2025] manages multi-region serving placement across clouds; SkyWalker [Xia et al., 2025] adds KV-aware cross-region load balancing with selective KV pushing. Inter-region round-trips routinely fall in the tens to hundreds of milliseconds [Vulimiri et al., 2015] and interact directly with TTFT when a request is forwarded across regions. **Production cross-region gateways.** Industrial cross-region routing exists but is opaque to KV state. AWS Bedrock cross-region inference [Amazon Web Services, 2026] and GKE multi-cluster Gateways [Google Cloud, 2026] provide global routing and failover but treat inference as an HTTP endpoint, with no surface for prefix locality or batching state. Serverless GPU platforms such as Modal [Modal, 2026] support intra-region prefix caching but do not optimize across regions or account for wide-area latency. SkyPilot [Yang et al., 2023] aggregates capacity across clouds but does not observe per-replica KV state. GORGO occupies the gap between these gateways and KV-aware single-region schedulers: a per-request cross-region routing layer that combines prefix locality, network RTT, and replica load on a single scalar score.

Table 1: Workload characterization. Token counts use the same byte-pair tokenizer in all rows. Reuse percentages are token-weighted prefix-trie savings as described in §3.

Dataset	Total reqs	Users	Avg input tokens	Intra-user reuse	Global reuse
LMSYS-Chat-1M	1,000,000	—	467	—	9.0%
WildChat-4.8M	3,199,860	1,833,730	2,925	5.3%	34.4%
GLM-5.1 production	411,169	4,984	21,043	53.7%	55.3%

3 Workload Characterization

Routing policies that perform well on short, low-reuse prompts may perform differently on long, high-reuse prompts because the prefill term in TTFT scales with uncached input tokens. We measure three datasets along the same axes: request length, conversational structure, and token-weighted prefix reuse.

Datasets. **LMSYS-Chat-1M** [Zheng et al., 2024a] is the Chatbot Arena dataset of one million conversations. **WildChat-4.8M** [Zhao et al., 2024] is a corpus of 3.2M ChatGPT-proxy conversations grouped by client IP. **GLM-5.1 production trace** is a 7-day chat-completion log from a long-context production endpoint [GLM Team, 2024]. It contains 411,169 requests across 4,984 distinct users, yielding $\sim$82 requests per user on average.

We measure prefix reuse with a token-weighted prefix trie that ingests each request's tokenized message sequence in arrival order, partitioned by user where the dataset carries user identity. The trie reports *intra-user savings* (fraction of tokens matching a prefix from the same user's prior requests), *cross-user savings* (additional fraction from pooling across users), and *global savings* (sum). LMSYS does not carry user identity, so we report only the global figure.

Three contrasts stand out (Table 1). *Request length*: GLM-5.1 prompts are 45$\times$ longer than LMSYS and 7$\times$ longer than WildChat. At typical prefill rates, a 21,000-token uncached prompt takes seconds. *User concentration*: GLM-5.1 has 82 requests per user on average; WildChat has 1.7; LMSYS has effectively one. Intra-user reuse depends on returning users, which the public sets simply do not have. *Reuse magnitude*: 53.7% of GLM-5.1 tokens are part of a prefix seen earlier from the same user (5.3% for WildChat, essentially zero for LMSYS). The bulk of WildChat's 34.4% global reuse comes from cross-user shared structure (chat templates, viral prompts), not sustained per-user dialogue.

For routing, LMSYS and WildChat live in a regime where prefix-aware policies have little to win. On GLM-5.1, a strategy that lands a user on a replica that has seen their conversation can save more than half of the prefill.

Experimental windows. We focus on two consecutive 30-minute windows of GLM-5.1 traffic: 00:30–01:00 UTC (W1, tuning) and 01:00–01:30 UTC (W2, held-out evaluation). Traces are replayed at original arrival speed and pre-filtered to 24,000 input tokens. Each window contains approximately 4,800 (W1) and 4,200 (W2) requests. Output is capped at 128 tokens so TTFT, not decode, dominates the latency budget.

Trace format and driver. The GLM-5.1 corpus is re-encoded into the Mooncake FAST'25 trace schema [Qin et al., 2024], preserving original arrival timing, so that every policy sees the same request order and inter-arrival pattern as production. The workload driver is open-loop: requests are issued at recorded arrival timestamps regardless of in-flight load, exposing each policy to the same burstiness as production rather than a closed-loop generator that would mask queueing pathologies.

4 GORGO

Let a request r with token sequence x_r be routed to replica u. Its TTFT decomposes as

$$\mathrm{TTFT}(r,u) = \mathrm{rtt}(u) + w_{\mathrm{queue}}(u,t_r) + t_{\mathrm{prefill}}(u)\cdot|x_r \setminus c_u(t_r)| + \varepsilon, \tag{1}$$

where $\mathrm{rtt}(u)$ is the round-trip from the routing proxy to u; $w_{\mathrm{queue}}(u,t_r)$ is queueing delay; $t_{\mathrm{prefill}}(u)$ is the per-token prefill rate; $c_u(t_r)$ is the set of token sequences cached on u at arrival time t_r; and ε

collects scheduler overhead and batching variance. Cross-region $\mathrm{rtt}(u)$ ranges from 26 ms to 466 ms in our cluster; the prefill term can be the largest and most routing-sensitive term at typical rates of 0.1–1 ms per uncached token on a 21,000-token prompt.

A GORGO replica score is a weighted sum of the three controllable terms in Equation 1:

$$\mathrm{score}(u, r) = \mathrm{rtt}_u + t_{\mathrm{prefill}} \cdot (|x_r| - |x_r \cap c_u|) + \alpha \cdot (\mathrm{queued\_tokens}_u + \mathrm{used\_kv\_tokens}_u). \quad (2)$$

The proxy sends r to the replica with the minimum score. Three signals feed the score: (i) rtt_u from the EWMA of a lightweight network probe decoupled from the metrics scrape; (ii) $|x_r \cap c_u|$ from the proxy's local radix trie of cached prefixes per replica, refreshed on every request completion; and (iii) load from the proxy's in-flight token counter combined with SGLang's reported KV-cache occupancy. The two free parameters are the per-token prefill rate t_{prefill} and the load weight α. Both parameters are stored *per replica*, with global defaults overridden on any replica whose GPU class, batch size, or model variant differs; the online tuner writes per-replica overrides automatically because each TTFT sample is labeled with its destination.

Three weight-selection variants. **gorgo-static** fixes t_{prefill} and α before the run and never adjusts them. In W1 we use manually chosen starter values ($t_{\mathrm{prefill}} = 0.07$, $\alpha = 0.06$). In W2, `gorgo-static` deploys the per-replica weights that `gorgo-hillclimb` ended W1 at ($t_{\mathrm{prefill}} \approx 0.983$, $\alpha \approx 4.4 \times 10^{-4}$) and never adjusts them again. This isolates the contribution of the cost model independent of online adaptation.

gorgo-autotune re-fits both parameters every 16 new request samples by taking the median of the observed rates $(\mathrm{TTFT} - \mathrm{rtt})/\mathrm{uncached\_tokens}$ from the trailing 64-sample window, partitioned by destination replica. It adapts to per-replica hardware drift and RTT shifts, but treats the cost-model weights as identifiable physical rates rather than black-box knobs.

gorgo-hillclimb runs a Gaussian $(1{+}1)$-evolution strategy [Rechenberg, 1973] over the same two parameters in log-space, with Rechenberg's 1/5-success rule controlling step size. The objective is the negative p95 TTFT of the rolling 64-request window. Every 16 new samples the tuner scores the current weights, accepts or rejects the previous candidate against the incumbent, and proposes a new candidate by perturbing each parameter as $\exp(\log(v) + \sigma\,\mathcal{N}(0, 1))$ with σ adapted according to the recent acceptance rate. This variant treats the cost-model weights as black-box knobs: it finds whatever values produce the best p95 TTFT under the current arrival distribution, with no offline profiling.

5 Baselines

We compare GORGO against five baseline policies.

random samples a replica uniformly per request; it is the lower bound.

least-request routes to the replica with the fewest in-flight requests, scoring by $\max(\mathrm{sgLang\_running\_reqs}_u, \mathrm{proxy\_inflight}_u)$ to bridge the $\sim$10 s staleness between metrics scrapes [F5 NGINX, 2024].

least-load routes to the replica with the lowest aggregate token-weighted load: $\mathrm{running} + \mathrm{queued} + \mathrm{used\_kv\_tokens} + \mathrm{queued\_tokens}_u$. It is heavier-signal than least-request but cache-blind.

prefix-cache is the AIBrix-style longest-prefix-match policy [AIBrix Team, 2024]. It picks the replica with the longest cached prefix match and falls back to least-request when the running-request imbalance exceeds a soft threshold.

simple-session-affinity hashes the first 256 token IDs of the prompt modulo the number of replicas. The same prefix always maps to the same replica, maximizing intra-user reuse but providing no load-escape mechanism.

6 Experimental Setup

Hardware and software. Each policy runs on its own dedicated three-replica fleet so routing decisions cannot warm or cool another policy's caches. Each fleet consists of one L40S:2 SGLang [Zheng et al., 2024b] server (tensor-parallel 2, 96 GB VRAM per replica) in each of Asia (Seoul),

Table 2: GLM-5.1 W1 (tuning window, 00:30–01:00 UTC). TTFT in seconds. Bold is best in column. Completed counts from proxy logs; success rate ≈84.2–84.6%.

Policy	Completed	p50	p95	p99	max	Succ.%
gorgo-hillclimb	4,065	**0.288**	**1.466**	**2.307**	4.39	84.4
simple-session-affinity	4,054	0.329	1.482	2.804	14.96	84.2
gorgo-static	4,076	0.299	1.605	2.357	4.16	84.6
least-request	4,074	0.293	1.632	2.355	13.71	84.6
prefix-cache	4,075	0.297	1.654	3.017	5.96	84.6
gorgo-autotune	4,076	0.294	1.658	2.428	4.18	84.6
least-load	4,066	0.297	1.728	2.629	4.64	84.4
random	4,076	0.296	1.828	3.171	28.99	84.6

Europe (Frankfurt), and US East (Ashburn), serving Qwen3.5-35B-A3B-FP8 [Qwen Team, 2025]. The model's native context window is 32,768 tokens; we cap workload input at 24,000 tokens to leave KV headroom for in-flight batches. A CPU proxy in US East hosts the routing policy and replays the trace.

The proxy scrapes Prometheus metrics every ∼10 s and runs a separate EWMA-smoothed HTTP probe for per-replica RTT, decoupled from metrics scrape to avoid handler contention. Before each run a homogeneity check issues 16 streaming requests directly to each replica; runs with worst-to-median TTFT ratio exceeding 3× are flagged (none occurred). **Cross-region RTT distribution.** EWMA-smoothed RTT from the US East proxy is 26–40 ms (US East), 90–130 ms (Frankfurt), and 260–466 ms (Seoul); the upper Seoul bound reflects routing-table churn during the trace, an 18× overall spread. **Policies and windows.** We evaluate eight policies (`random`, `least-request`, `least-load`, `prefix-cache`, `simple-session-affinity`, `gorgo-static`, `gorgo-autotune`, `gorgo-hillclimb`) in each window. All eight fleets start at the same wall-clock time on the same input trace at concurrency 32. W1 seeds GORGO variants with manually chosen weights ($t_{\text{prefill}} = 0.07$, $\alpha = 0.06$); W2 seeds them with the per-replica values that `gorgo-hillclimb` learned during W1 ($t_{\text{prefill}} \approx 0.983$, $\alpha \approx 4.4 \times 10^{-4}$). `gorgo-static` in W2 deploys those weights without adjusting; the others continue tuning on the held-out window. **Metrics.** We report TTFT p50, p95, and p99 over all successful streaming responses. Failed requests (HTTP 4xx/5xx, dominated by prompt-too-long at the effective context boundary) are excluded from percentiles but counted in success rate; success rate is 84.2–85.1% across all policies and windows.

7 Results

In W1 (Table 2), `gorgo-hillclimb` delivers the lowest p50 (0.288 s), p95 (1.466 s), and p99 (2.307 s). The next-best p95 is `simple-session-affinity` at 1.482 s, a 1.1% gap. The `gorgo-static` configuration—with manually-set starting weights, never adjusted—already beats every non-GORGO policy on p99 and ranks third on p95, showing that the cost model alone improves on single-signal heuristics. The p95 spread across all policies in W1 is 1.25× (1.466 s vs. 1.828 s).

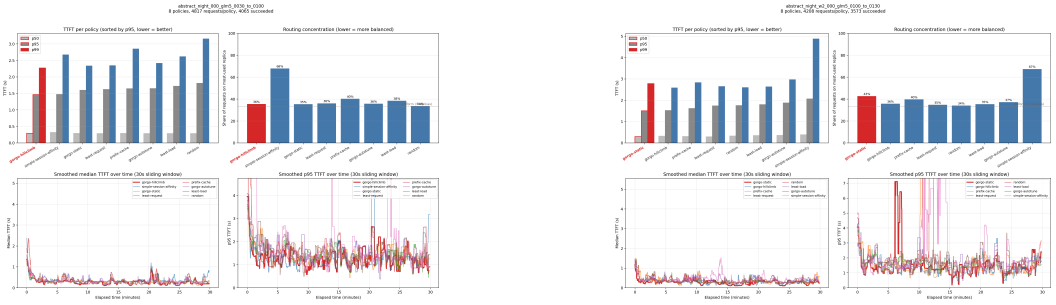
`prefix-cache` is competitive at p95 (1.654 s) but tail-degraded at p99 (3.017 s): it concentrates traffic onto warm-cache replicas and pays a queueing penalty when one is momentarily saturated. `simple-session-affinity` achieves the second-best p95 from per-user temporal locality but its p99 (2.804 s) and max (14.96 s) reflect episodes where the hashed replica is transiently overloaded with no escape mechanism. Starting from $t_{\text{prefill}} = 0.07$, $\alpha = 0.06$, the ES converged to $t_{\text{prefill}} \approx 0.983$ and $\alpha \approx 4.4 \times 10^{-4}$: the high t_{prefill} reflects long uncached prefixes (∼9.5k tokens after 55% reuse on 21k-token prompts), and the low α reflects that SGLang's continuous batching already orders within a replica and deep queues are rare at concurrency 32 on this arrival rate.

In W2 (Table 3), `gorgo-static`—deploying the W1-learned weights without adjustment—achieves the best p50 (0.294 s) and p95 (1.510 s). `gorgo-hillclimb` achieves the best p99 (2.595 s). The two GORGO variants together sweep all three percentiles; no single GORGO variant does. The p95 spread in W2 is 1.38×, wider than W1's 1.25×.

`gorgo-static` beats `gorgo-hillclimb` on p50/p95 in W2 because the W1-learned weights are near-optimal for the W2 distribution; continuing to tune adds proposal variance, a known prop-

Table 3: GLM-5.1 W2 (held-out evaluation window, 01:00–01:30 UTC). `gorgo-static` deploys the W1-learned weights and does not adjust them. Bold is best in column.

Policy	Completed	p50	p95	p99	max	Succ.%
gorgo-static	3,573	**0.294**	**1.510**	2.791	15.26	84.9
gorgo-hillclimb	3,583	0.334	1.535	**2.595**	12.68	85.1
prefix-cache	3,578	0.316	1.632	2.860	34.34	85.0
least-request	3,579	0.303	1.757	2.663	5.22	85.1
random	3,575	0.336	1.763	2.612	30.93	85.0
least-load	3,580	0.363	1.813	2.659	3.84	85.1
gorgo-autotune	3,578	0.368	1.888	3.001	5.70	85.0
simple-session-affinity	3,570	0.396	2.080	4.917	36.60	84.8



(a) W1 (tuning window). (b) W2 (held-out evaluation window).

Figure 1: TTFT distributions for all eight policies on the GLM-5.1 trace. `gorgo-hillclimb` sweeps p50/p95/p99 in W1; in W2 `gorgo-static` (W1-learned weights, frozen) leads at p50/p95 and `gorgo-hillclimb` leads at p99.

erty of $(1{+}1)$-ES under stationary objectives. The practical takeaway is that operators can run `gorgo-hillclimb` during a tuning window and freeze the result into `gorgo-static` for production. `gorgo-autotune`'s median-of-rates fit drifts when uncached counts are small (high cache hit), which the black-box ES sidesteps by integrating over the mixture rather than disentangling it. `simple-session-affinity`'s p95 inflates 40.4% and its p99 by 75.4% from W1 to W2—a hash-to-replica mapping cannot rebalance when a bucket spikes—whereas `gorgo-static`'s W2 p95 improves 6.0% over W1 and is the only policy whose p95 improves across windows.

8 Limitations

Statistical scope. Two consecutive 30-minute windows of one production trace share model, fleet, hour-of-day, and arrival-rate band: W2 demonstrates held-out generalization within that workload, not arbitrary transfer across hours, days, or models. Each policy is run once per window. With $\sim$4,200–4,800 successful responses, p99 has approximate standard error $\sigma_{\mathrm{TTFT}}/\sqrt{n \cdot 0.01 \cdot 0.99} \approx 0.05$ s, so the W2 p99 spread between `gorgo-hillclimb` (2.595 s) and `random` (2.612 s) is 0.017 s—inside one standard error and not a reliable advantage; the W1 sweep and W2 p50/p95 wins are several standard errors clear of zero. Replication across windows for confidence intervals on every percentile is left to future work. **Operational scope.** Each policy ran on its own dedicated three-replica fleet, so cache state is single-tenant; multi-tenant sharing changes the queue and cache dynamics that GORGO's load weight captures and is not validated here. The routing proxy is centralized and is therefore a single point of failure; a distributed GORGO variant using peer-summary coordination is sketched in our design but not benchmarked, and we expect distributed coordination to win for proxy-bandwidth-bound regimes that this evaluation does not reach. **Methodological caveats.** `gorgo-autotune` is the weakest GORGO variant in W2: its median-of-rates fit treats cost-model weights as identifiable physical rates, but the rate $(\mathrm{TTFT} - \mathrm{rtt})/\mathrm{uncached}$ inflates when uncached counts are small (high cache hit), and the variant drifts—a known failure mode of identifiability-style fits relative to black-box optimization. The $(1{+}1)$-ES underlying `gorgo-hillclimb` requires several hop cycles to converge from a random starting point; in W1 this fits inside 30 minutes, but very short

traffic windows may not. **System scope.** All replicas use identical GPU configurations; the fleet does not split prefill and decode [Zhong et al., 2024, Patel et al., 2024], which would require an additional routing term. GORGO uses only metrics exposed by an HTTP-fronted SGLang/vLLM server, not scheduler-internal information [Pan et al., 2025, Yang et al., 2025a] that would tighten the prefill estimate.

9 Related Work

Prefix caches and reuse. RadixAttention in SGLang [Zheng et al., 2024b] stores per-request KV state in a radix tree; PagedAttention in vLLM [Kwon et al., 2023] enables prefix sharing through paged memory. KVLink [Yang et al., 2025b], ChunkKV [Liu et al., 2025], KVFlow [Pan et al., 2025], and Learned Prefix Caching [Yang et al., 2025a] improve the single-replica cache but do not decide which replica serves a request. **Routing and scheduling.** Mooncake [Qin et al., 2024] is a KV-cache-centric architecture and the source of our trace format. AIBrix [AIBrix Team, 2024] provides prefix-aware load balancing as a baseline; k-LPM [Dexter et al., 2025] formulates LLM scheduling under TTFT constraints as NP-hard; DLPM [Cao et al., 2025] targets fairness with locality; Llumnix [Sun et al., 2024] migrates requests across replicas; Multi-LLM routers [Wu and Silwal, 2025] address model selection; CacheBlend [Yao et al., 2024] routes by trie-measured prefix length but does not measure network RTT. GORGO commits to a single scalar score per replica with online weight tuning and requires no engine modifications. **Online and learned routing.** Performance-aware LLM load balancing [Jain et al., 2025] learns a routing policy over mixed prefill/decode workloads; bandit and RL schedulers such as Decima [Mao et al., 2019] adapt placement from observed outcomes; CherryPick [Alipourfard et al., 2017] uses Bayesian optimization to tune cloud configurations from a small parameter space. GORGO keeps the policy interpretable (an additive cost) and applies $(1{+}1)$-ES [Rechenberg, 1973] only to its two scaling weights, with TTFT as direct reward—fast and overhead-free with no surrogate model. **PD-disaggregation.** Sarathi-Serve [Agrawal et al., 2024] interleaves chunked prefill with decode at the batch level; DistServe [Zhong et al., 2024] and Splitwise [Patel et al., 2024] dedicate separate GPU pools to prefill and decode. These reduce the compute component of TTFT but do not resolve geo-routing trade-offs when locality and network latency conflict; GORGO is complementary and assumes an underlying stack that may already use either. **Classic load balancing.** Power-of-two-choices [Mitzenmacher, 2001] and consistent-hash affinity [Karger et al., 1997] underpin production L7 balancers, including the prefix-aware AIBrix-style routers we benchmark against [AIBrix Team, 2024, F5 NGINX, 2024]. Their coordination assumes microsecond-scale homogeneous backends, whereas GPU LLM replicas process requests for seconds and per-token prefill rates differ by an order of magnitude depending on locality.

10 Conclusion

We characterize a workload regime—long-context, multi-turn, high prefix-reuse production traffic—where standard LLM-serving routing heuristics leave performance on the table because they pick one signal out of three (cache locality, replica load, network cost). On a 411,000-request production trace where 53.7% of input tokens are part of a previously-seen prefix from the same user, three variants of a joint cost model collectively achieve the lowest TTFT at every percentile (p50, p95, p99) on both a tuning and a held-out evaluation window, against seven baseline policies, on a multi-region L40S:2 fleet with cross-region RTTs from 26 to 466 ms. Per-percentile wins are not held by any single fixed weight setting; the best evaluation-window configuration is the one that froze the weights learned in the tuning window. The cost model and tuner fit on a stock HTTP gateway and require no offline profiling.

References

A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, R. Ramjee, and A. Tumanov. Sarathi-Serve: Llm inference made efficient and fair. arXiv preprint, 2024.

AIBrix Team. AIBrix: An open-source infrastructure for LLM inference at scale. `https://github.com/vllm-project/aibrix`, 2024.

Omid Alipourfard, Hongqiang Harry Liu, Jianshu Chen, Shivaram Venkataraman, Minlan Yu, and Ming Zhang. CherryPick: Adaptively unearthing the best cloud configurations for big data

analytics. In *Proceedings of the 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI '17)*, 2017.

Amazon Web Services. Increase throughput with cross-region inference. Documentation, 2026. Accessed 2026-01-28.

Shiyi Cao, Yichuan Wang, Ziming Mao, Pin-Lun Hsu, Liangsheng Yin, Tian Xia, Dacheng Li, Shu Liu, Yineng Zhang, Yang Zhou, Ying Sheng, Joseph Gonzalez, and Ion Stoica. Locality-aware fair scheduling in LLM serving. *arXiv preprint arXiv:2501.14312*, 2025.

Gregory Dexter, Shao Tang, Ata Fatahi, Qingquan Song, Tejas Dharamsi, and Aman Gupta. LLM query scheduling with prefix reuse and latency constraints. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

F5 NGINX. NGINX documentation: Load balancing methods. `https://docs.nginx.com/nginx/admin-guide/load-balancer/http-load-balancer/`, 2024.

GLM Team. GLM-4: A family of open multilingual multimodal chat LLMs, 2024.

Google Cloud. About multi-cluster gateways. Documentation, 2026. Accessed 2026-01-28.

Kunal Jain, Anjaly Parayil, Ankur Mallick, Esha Choukse, Xiaoting Qin, Jue Zhang, Íñigo Goiri, Rujia Wang, Chetan Bansal, Victor Rühle, Anoop Kulkarni, Steve Kofsky, and Saravan Rajmohan. Performance aware LLM load balancer for mixed workloads. In *Proceedings of the 5th Workshop on Machine Learning and Systems (EuroMLSys '25)*, 2025.

David Karger, Eric Lehman, Tom Leighton, Rina Panigrahy, Matthew Levine, and Daniel Lewin. Consistent hashing and random trees: Distributed caching protocols for relieving hot spots on the world wide web. In *Proceedings of the 29th Annual ACM Symposium on Theory of Computing (STOC '97)*, 1997.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2023.

Xiang Liu, Zhenheng Tang, Peijie Dong, Zeyu Li, Yue Liu, Bo Li, Xuming Hu, and Xiaowen Chu. ChunkKV: Semantic-preserving KV cache compression for efficient long-context LLM inference. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

H. Mao, Z. Yang, A. Phanishayee, R. Mahajan, and I. Stoica. SkyServe: Serving ai models across regions and clouds with spot instances. In *Proceedings of the 2025 USENIX Annual Technical Conference*, 2025.

Hongzi Mao, Malte Schwarzkopf, Shaileshh Bojja Venkatakrishnan, Zili Meng, and Mohammad Alizadeh. Learning scheduling algorithms for data processing clusters. In *Proceedings of the ACM Special Interest Group on Data Communication (SIGCOMM '19)*, 2019.

Michael Mitzenmacher. The power of two choices in randomized load balancing. *IEEE Transactions on Parallel and Distributed Systems*, 12(10):1094–1104, 2001.

Modal. Prefix caching. Documentation, 2026. Accessed 2026-01-28.

Zaifeng Pan, Ajjkumar Patel, Zhengding Hu, Yipeng Shen, Yue Guan, Wan-Lu Li, Lianhui Qin, Yida Wang, and Yufei Ding. KVFlow: Efficient prefix caching for accelerating LLM-based multi-agent workflows. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2507.07400.

Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2024.

Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yanzhe Wu, Weimin Zheng, Xinran Chen, Rui Xu, Xingda Yao, Rongxin Wei, Zhiyuan Zhao, Jing Huang, Mingjie Yang, and Jidong Wu. Mooncake: A KVCache-centric disaggregated architecture for LLM serving, 2024.

Qwen Team. Qwen3 technical report. 2025. Alibaba Cloud.

Ingo Rechenberg. Evolutionsstrategie: Optimierung technischer systeme nach prinzipien der biologischen evolution. 1973.

Varun Srivatsa, H. Gao, M. Patel, A. Sivaraman, and A. Rastogi. Preble: Efficient distributed prompt scheduling for large language model serving. arXiv preprint arXiv:2407.00023, 2024.

Biao Sun, Ziming Huang, Hanyu Tang, Chengquan Wang, Cheng Zhang, Yiming Li, Liu Yang, Wencong Zhang, Lin-Wei Wang, Tianhe Wu, Ang Cao, Yongqiang Lin, et al. Llumnix: Dynamic scheduling for large language model serving. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

Ashish Vulimiri, Carlo Curino, P. Brighten Godfrey, Thomas Jungblut, Jitendra Padhye, and George Varghese. Global analytics in the face of bandwidth and regulatory constraints. In *Proceedings of the 12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*, 2015.

Fangzhou Wu and Sandeep Silwal. Efficient training-free online routing for high-volume multi-LLM serving. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

Tian Xia, Ziming Mao, Jamison Kerney, Ethan J. Jackson, Zhifei Li, Jiarong Xing, Scott Shenker, and Ion Stoica. SkyWalker: A locality-aware cross-region load balancer for LLM inference. arXiv preprint arXiv:2505.24095, 2025.

Dongsheng Yang, Austin Li, Kai Li, and Wyatt Lloyd. Learned prefix caching for efficient LLM inference. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025a.

Jingbo Yang, Bairu Hou, Wei Wei, Yujia Bao, and Shiyu Chang. KVLink: Accelerating large language models via efficient KV cache reuse. *arXiv preprint arXiv:2502.16002*, 2025b.

Zongheng Yang et al. SkyPilot: An intercloud broker for sky computing. Preprint, 2023.

Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Peng, Hao Zhang, Ion Stoica, Kurt Keutzer, and Michael W. Mahoney. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion, 2024.

Gyeong-In Yu, Jinseok Jeong, Gyeongchan Kim, Suhyun Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022.

Wenting Zhao, Xiang Ren, Jack Hessel, Claire Cardie, Yejin Choi, and Yuntian Deng. WildChat: 1M ChatGPT interaction logs in the wild. In *International Conference on Learning Representations (ICLR)*, 2024.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Tianle Li, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zhuohan Li, Zi Lin, Eric P. Xing, Joseph E. Gonzalez, Ion Stoica, and Hao Zhang. LMSYS-Chat-1M: A large-scale real-world LLM conversation dataset. In *International Conference on Learning Representations (ICLR)*, 2024a.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024b.

Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

NeurIPS Paper Checklist

1. **Claims**
   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?
   Answer: [Yes]
   Justification: The abstract claims (i) public datasets underrepresent long-context multi-turn traffic (quantified in §3), (ii) we evaluate routing policies on a production trace (Tables 2 and 3), and (iii) GORGO sweeps every TTFT percentile in both windows. We are explicit in the introduction and §8 that "GORGO" refers to a family and that the W2 p99 margin is at the noise floor.

2. **Limitations**
   Question: Does the paper discuss the limitations of the work?
   Answer: [Yes]
   Justification: Section 8 groups limitations into statistical scope (single trace, single run per window with derived p99 standard error), operational scope (single-tenant fleet, centralized-proxy SPOF), methodological caveats (`gorgo-autotune` identifiability failure, ES convergence time), and system scope (homogeneous hardware, no PD-disaggregation, no scheduler-internal signals).

3. **Theory assumptions and proofs**
   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete proof?
   Answer: [N/A]
   Justification: The paper presents an additive cost model and an online optimizer with no formal theorems.

4. **Experimental result reproducibility**
   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results?
   Answer: [Yes]
   Justification: Section 6 specifies model, regions, hardware, fleet topology, concurrency, trace windows, and per-window GORGO seeding. Section 4 gives Equation 2, the $(1{+}1)$-ES configuration ($\sigma_0 = 0.5$, 1/5-rule, hop 16, window 64), and the median-of-rates fit. The reproducibility statement enumerates four canonical paths in the released artifact.

5. **Open access to data and code**
   Question: Does the paper provide open access to the data and code?
   Answer: [No]
   Justification: The GLM-5.1 production trace is not redistributable under its terms of service. The proxy and policy code is not yet open-sourced at submission time. The public-dataset characterization (LMSYS, WildChat) is fully reproducible with no proprietary inputs.

6. **Experimental setting/details**
   Question: Does the paper specify all the training and test details?
   Answer: [Yes]
   Justification: Section 6 specifies model, hardware, regions, concurrency, trace windows, max input tokens, and per-window seeding for adaptive policies. ES configuration is given in Section 4.

7. **Experiment statistical significance**
   Question: Does the paper report error bars or appropriate statistical significance information?
   Answer: [No]
   Justification: Each data point is a single 30-minute window. Section 8 derives an approximate p99 standard error and shows that the W2 p99 spread between `gorgo-hillclimb` and `random` is inside one standard error. The W1 sweep and W2 p50/p95 wins are several standard errors clear of zero.

8. **Experiments compute resources**
   Question: Does the paper provide sufficient information on compute resources?
   Answer: [Yes]
   Justification: Each policy runs on a dedicated 3-replica fleet of L40S:2 SGLang servers

(tensor-parallel 2, 96 GB VRAM per replica). Eight policies $\times$ two windows $= 16$ fleet-runs, each $\sim$30 min wall clock, on three regions. Total GPU-hours: $\approx$24.

9. **Code of ethics**
   Question: Does the research conform with the NeurIPS Code of Ethics?
   Answer: [Yes]
   Justification: The work concerns routing infrastructure. The production trace is used under terms that permit research replay; we release only aggregated prefix-trie statistics, not individual requests. No individuals are identifiable from the released aggregates.

10. **Broader impacts**
    Question: Does the paper discuss potential societal impacts?
    Answer: [Yes]
    Justification: Routing improvements reduce the energy and dollar cost of serving LLMs at fixed quality. Lower-cost serving may accelerate deployment in settings where that is socially harmful; the contributions are infrastructure-level and do not affect model capability.

11. **Safeguards**
    Question: Does the paper describe safeguards for responsible release?
    Answer: [N/A]
    Justification: We do not release datasets or models. The released code is a routing proxy and policy library with no inherent dual-use beyond LLM-serving infrastructure.

12. **Licenses**
    Question: Are creators or owners of assets properly credited?
    Answer: [Yes]
    Justification: SGLang, vLLM, AIBrix, LMSYS-Chat-1M, WildChat-4.8M, and Qwen3 are cited and used under their published licenses.

13. **New assets**
    Question: Are new assets introduced in the paper well documented?
    Answer: [Yes]
    Justification: The proxy, policy library, experiment controller, and trace builder are documented in the repository's top-level documentation. Spec and manifest files are stored alongside the controller.

14. **Crowdsourcing and research with human subjects**
    Question: Does the paper involve crowdsourcing or human subjects?
    Answer: [N/A]
    Justification: No human subjects.

15. **IRB approvals**
    Answer: [N/A]
    Justification: No human subjects.

16. **Declaration of LLM usage**
    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the research methodology?
    Answer: [N/A]
    Justification: LLM serving infrastructure is the object of study. Authors used LLMs for routine writing assistance only.

Reproducibility Statement

The experiments are driven by a single experiment controller that launches fleets, configures policies, and replays a trace per policy. The four canonical paths in the released artifact are:

- `specs/policy_matrix_abstract_night.json` — W1 spec (eight policies, manual GORGO starter weights);
- `specs/policy_matrix_abstract_night_w2.json` — W2 spec (eight policies, GORGO seeded with W1-learned weights);
- `experiment_runner/policy_matrix_app.py` — experiment controller;
- `data_processing/build_mooncake_trace.py` — trace builder.

W1 and W2 manifests, result CSVs, and summary plots are produced by running the controller against these specs and post-processing per-policy stats with the released analysis scripts. The GLM-5.1 production trace is not redistributable; the public-dataset characterization (LMSYS, WildChat) reproduces with no proprietary inputs.